

Error Handling in React

Choose the handler that matches where a failure occurs. Give users a useful recovery path while keeping implementation details out of the interface.

Match the Failure

Failure	Handler
Component throws while rendering	React Error Boundary
Route loader, action, or route rendering fails	React Router <code>errorElement</code>
Event handler or asynchronous task fails	<code>try/catch</code> where the work starts
Invalid user input	Validation and field-level feedback
HTTP request returns a failure status	Check <code>response.ok</code> and handle it

A React Error Boundary does not catch errors from event handlers, timers, or asynchronous callbacks. Handle those at their source. It also cannot catch errors thrown by the boundary itself.

React Error Boundary

A boundary catches errors in descendant rendering, constructors, and lifecycle methods. This app's top-level boundary replaces a broken screen with a recovery message and logs diagnostic details.

```
import { Component, type ErrorInfo, type ReactNode } from "react";

type Props = { children: ReactNode };
type State = { hasError: boolean };

class AppErrorBoundary extends Component<Props, State> {
  state: State = { hasError: false };

  static getDerivedStateFromError(): State {
    return { hasError: true };
  }

  componentDidCatch(error: Error, info: ErrorInfo) {
    console.error("Uncaught rendering error", error, info.componentStack);
  }

  render() {
    if (this.state.hasError) {
      return <p role="alert">Something went wrong. Reload to try again.</p>;
    }
    return this.props.children;
  }
}
```

`getDerivedStateFromError` selects the fallback UI. `componentDidCatch` is for diagnostics or reporting to a monitoring service. Do not expose stack traces or sensitive error details to users.

React Router Route Errors

Data routers render the nearest route error boundary when a loader, action, or route component fails. Use `useRouteError` to read the error and `isRouteErrorResponse` to identify expected route response errors.

```
import { isRouteErrorResponse, useRouteError } from "react-router";

function RouteErrorPage() {
  const error = useRouteError();
  if (isRouteErrorResponse(error)) {
    return <h1>{error.status}: {error.statusText}</h1>;
  }
  return <h1>We could not load this page.</h1>;
}

const router = createBrowserRouter([
  {
    path: "/",
    element: <HomePage />,
    loader: loadHomeData,
    errorElement: <RouteErrorPage />,
  },
]);
```

Use route boundaries for navigation-level recovery. Add a wildcard route for unmatched URLs and give users a clear destination, such as returning home.

Async and Event Errors

Promise rejections must be caught by the async task. Also note that `fetch` resolves for HTTP errors such as 404 and 500, so check `response.ok`.

```
async function loadProfile() {
  try {
    const response = await fetch("/api/profile");
    if (!response.ok) {
      throw new Error(`Request failed: ${response.status}`);
    }
    const profile = await response.json();
    setProfile(profile);
    setError(null);
  } catch {
    setError("Profile could not be loaded. Please try again.");
  }
}
```

For expected network failures, show loading and error states and preserve user data where possible. Save Error Boundaries for unexpected failures that prevent part of the UI from rendering.

Try the App Demo

- Choose **Trigger a loader error** to view the React Router route error screen.
- Visit an unknown URL to view the not-found page.
- Rendering errors below the router are handled by the top-level React Error Boundary.